CPU Scheduling Simulator

A Comparative Analysis of
Round Robin · SJF (Non-Preemptive) · SRTF (Preemptive SJF)

Operating Systems Course — Scheduling Comparison Project
C5 | Academic Report

Table of Contents

1. Introduction

CPU scheduling is the mechanism by which an operating system decides which process in the ready queue should be allocated the CPU at any given moment. It is one of the most fundamental components of OS design, directly influencing system throughput, process responsiveness, resource utilisation, and overall user experience. A well-chosen scheduling algorithm can dramatically improve the efficiency of a system; a poorly-chosen one can lead to starvation, poor response times, and wasted CPU cycles.

This report presents a detailed comparative analysis of three CPU scheduling algorithms: **Round Robin (RR)**, **Shortest Job First — Non-Preemptive (SJF)**, and **Shortest Remaining Time First — Preemptive SJF (SRTF)**. The analysis is based on a Python simulator with an interactive Tkinter GUI that runs all three algorithms on identical process datasets and presents per-process metrics (Waiting Time, Turnaround Time, Response Time), Gantt charts, and a full comparison report.

The key evaluation dimensions are:

- **Performance** — which algorithm minimises waiting and turnaround times
- **Fairness** — how equitably CPU time is distributed across processes
- **Responsiveness** — how quickly a process first receives the CPU
- **Starvation** — whether any process can be indefinitely delayed
- **Quantum effect** — how the RR time quantum influences all metrics

2. Algorithm Overview

2.1 Round Robin (RR)

Round Robin is the standard CPU scheduling algorithm for time-sharing systems. Each process is assigned a fixed time quantum (time slice). The ready queue is maintained as a circular FIFO: the front process runs for up to one quantum, then is moved to the back, and the next process runs. This continues until every process has completed.

Key properties:

- Preemptive — a running process is interrupted after each quantum expires
- Fair — every process receives CPU time within at most (n–1)×quantum time units
- No starvation — every process in the queue is guaranteed eventual service
- Response time bounded by the quantum size
- Context-switch overhead increases as quantum decreases
- Performance degrades toward FCFS as quantum grows large

When to use: Interactive operating systems, time-sharing environments, any system where equal CPU sharing and bounded response times are priorities.

2.2 Shortest Job First — Non-Preemptive (SJF)

SJF (Non-Preemptive) selects the process with the smallest burst time from the set of currently arrived, unfinished processes whenever the CPU becomes free. Once a process starts executing, it runs to completion — it cannot be interrupted. Ties in burst time are broken by arrival time, then by process ID.

Key properties:

- Non-preemptive — a running process cannot be interrupted
- Optimal for minimising average waiting time among non-preemptive algorithms
- Starvation possible — long jobs may wait indefinitely if short jobs keep arriving
- Requires knowledge of burst times in advance (impractical in its pure form)
- Produces a Gantt chart where each bar spans the full burst of a process
- High response time for long processes competing with many short ones

When to use: Batch processing environments where burst times are known in advance and starvation of individual jobs is not a critical concern.

2.3 SRTF — Shortest Remaining Time First (Preemptive SJF)

SRTF is the preemptive extension of SJF. At every time unit, the scheduler inspects all available processes and runs the one with the shortest *remaining* burst time. If a newly arrived process has a shorter remaining time than the currently running process, the running process is immediately preempted and the new process takes the CPU.

Key properties:

- Preemptive — a running process can be interrupted at any time unit
- Theoretically optimal — minimises average waiting time among all preemptive policies
- Highest starvation risk — long jobs may never receive CPU if short jobs stream in
- Produces the most fragmented Gantt chart due to frequent preemptions
- Context-switch overhead is the highest of the three algorithms
- Response time for short processes is extremely low

When to use: Batch systems where minimising average waiting time is paramount, preemption is allowed, and burst-time estimates are available.

3. Implementation Overview

The simulator is implemented as a single Python file (**scheduler.py**) using the Tkinter GUI toolkit. The code is organised into clearly separated layers:

- **Process dataclass** — immutable input fields (pid, arrival, burst) plus computed scheduling metrics (WT, TAT, RT, finish time). A clone() method provides a fresh copy for each algorithm run.
- **Scheduling functions** — schedule_rr(), schedule_sjf(), schedule_srtf() each return (procs, gantt) tuples. The RR function additionally returns rq_history for the Ready Queue View panel.
- **Metrics helpers** — compute_averages() and fairness_index() (Jain's) operate on any list of scheduled Process objects.
- **Gantt chart renderer** — draw_gantt() uses a Tkinter Canvas with a cumulative timeline: every block's x-coordinate is computed as MARGIN + time × scale, guaranteeing pixel-perfect contiguity.
- **GUI** — SchedulerApp (tk.Tk) with a left input panel and a four-tab notebook (RR | SJF | SRTF | Comparison). Input validation rejects negative times, zero bursts, duplicate PIDs, and invalid quantum values.
- **Comparison generator** — generate_comparison() produces a structured textual report with six analysis questions, long-process treatment, SRTF-specific observations, and a workload recommendation.

Input Validation Rules

Rule	Constraint
Process ID	Non-empty string; must be unique
Arrival Time	Non-negative integer (≥ 0)
Burst Time	Strictly positive integer (> 0)
Time Quantum (RR)	Strictly positive integer (> 0)

4. Test Datasets

Five test scenarios are built into the simulator. All algorithms always run on the *same* process set so results are directly comparable.

Scenario A: Basic Mixed Workload

A general-purpose workload with varied burst times and staggered arrivals. Suitable for observing all three algorithms under typical conditions.

PID	Arrival Time	Burst Time
P1	0	10
P2	1	4
P3	2	6
P4	3	2
P5	4	8

Scenario B: Short-Job-Heavy

Several very short processes alongside one long process (P5). Makes SJF and SRTF's preference for short jobs highly visible and demonstrates the starvation risk for P5.

PID	Arrival Time	Burst Time
P1	0	2
P2	0	3
P3	1	1
P4	2	2
P5	3	20

Scenario C: Fairness Case (RR Advantage)

All processes arrive simultaneously with identical burst times. RR distributes CPU perfectly evenly; SJF and SRTF are equivalent to FCFS in this scenario.

PID	Arrival Time	Burst Time
P1	0	12
P2	0	12
P3	0	12
P4	0	12

Scenario D: Long-Job Sensitivity

One very long process (P1) competes with shorter ones. Illustrates how SJF and SRTF penalise P1 in favour of shorter jobs.

PID	Arrival Time	Burst Time
P1	0	30
P2	1	4
P3	2	6
P4	5	2

Scenario E: SRTF Preemption Showcase

Designed so P2 preempts P1 at t=1, making SRTF preemption unambiguous in the Gantt chart. P1 is interrupted after just 1 unit.

PID	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

5. Simulation Results

Results below are for **Scenario A — Basic Mixed Workload** (5 processes, quantum = 3). This is the primary comparison dataset used throughout the report. Scenario E results are shown separately to highlight SRTF preemption behaviour.

5.1 Basic Mixed Workload

Round Robin (q=3)

PID	Arrival	Burst	Wait Time	Turnaround	Response
P1	0	10	18	28	0
P2	1	4	13	17	2
P3	2	6	13	19	4
P4	3	2	6	8	6
P5	4	8	18	26	10

Avg WT: **13.60** Avg TAT: **19.60** Avg RT: **4.40**

SJF — Non-Preemptive

PID	Arrival	Burst	Wait Time	Turnaround	Response
P1	0	10	0	10	0
P2	1	4	11	15	11
P3	2	6	14	20	14
P4	3	2	7	9	7
P5	4	8	18	26	18

Avg WT: **10.00** Avg TAT: **16.00** Avg RT: **10.00**

SRTF — Preemptive SJF

PID	Arrival	Burst	Wait Time	Turnaround	Response
P1	0	10	20	30	0
P2	1	4	0	4	0
P3	2	6	5	11	5
P4	3	2	2	4	2

PID	Arrival	Burst	Wait Time	Turnaround	Response
P5	4	8	9	17	9

Avg WT: **7.20** Avg TAT: **13.20** Avg RT: **3.20**

Three-Algorithm Average Comparison

Metric	Round Robin	SJF (Non-Preemptive)	SRTF (Preemptive)
Avg Wait Time	13.60	10.00	★ 7.20
Avg Turnaround	19.60	16.00	★ 13.20
Avg Response	4.40	10.00	★ 3.20

★ marks the best (lowest) value in each row. SRTF achieves the lowest average waiting time, consistent with its theoretical optimality. RR achieves the lowest average response time due to its quantum-bounded first-CPU-access guarantee.

5.2 SRTF Preemption Showcase (Scenario E)

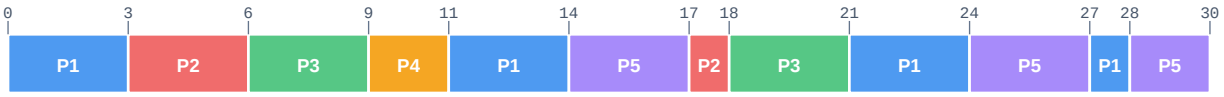
This dataset is specifically designed to make preemption unmistakable. P1 starts at t=0 (burst=8). P2 arrives at t=1 with burst=4 — shorter than P1's remaining time of 7 — so P1 is immediately preempted. The Gantt chart below shows P1 running for just 1 unit before P2 takes over.

PID	Arrival	Burst	Wait Time	Turnaround	Response
P1	0	8	9	17	0
P2	1	4	0	4	0
P3	2	9	15	24	15
P4	3	5	2	7	2

6. Gantt Chart Analysis

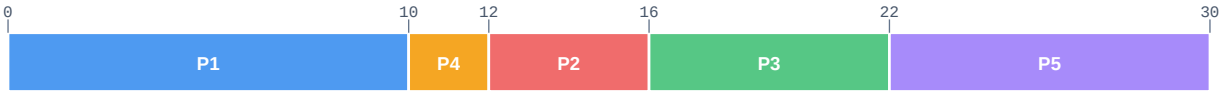
All Gantt charts use a **cumulative timeline**: time never resets between segments. Each bar's width is directly proportional to (end – start), and adjacent bars share an exact pixel boundary. Idle CPU periods are shown as grey IDLE blocks.

Round Robin (q=3) — Basic Mixed Workload



RR produces the most segments because every process is sliced at the quantum boundary. Notice how all five processes interleave — no process runs for more than 3 time units without yielding. This circular rotation is precisely what gives RR its fairness and bounded response time.

SJF Non-Preemptive — Basic Mixed Workload



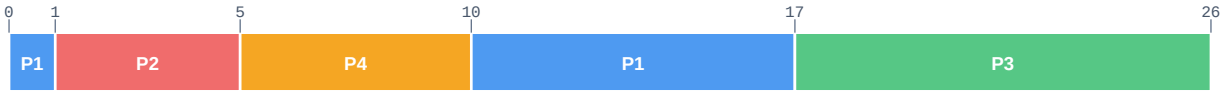
SJF produces the fewest, longest bars because each process runs to completion without interruption. The execution order reflects burst-length ranking among arrived processes — shorter jobs are served before longer ones whenever the CPU becomes free, even if they arrived later.

SRTF — Basic Mixed Workload

SRTF shows intermediate fragmentation — more segments than SJF (preemptions occur each time a shorter job arrives) but fewer than RR (preemption only happens on a new arrival, not on a fixed clock). Each split in a process bar marks an

exact preemption point.

SRTF — Preemption Showcase (Scenario E)



P1 runs for exactly 1 unit (t=0→1), then P2 preempts it at t=1. P2 completes at t=5, P4 (burst=5, remaining shortest) runs t=5→10, then P1 resumes with remaining=7 at t=10. P3 (burst=9) runs last. This Gantt chart makes SRTF preemption entirely unambiguous.

7. Comparative Analysis

7.1 Performance Comparison (Basic Mixed Workload)

Metric	Round Robin	SJF (Non-Preemptive)	SRTF (Preemptive)
Avg Wait Time	13.60	10.00	★ 7.20
Avg Turnaround	19.60	16.00	★ 13.20
Avg Response	4.40	10.00	★ 3.20

7.2 Algorithm Trade-offs

Property	Round Robin	SJF (Non-Preemptive)	SRTF (Preemptive)
Preemptive?	Yes (time-based)	No	Yes (length-based)
Avg Wait Time	Moderate	Near-optimal	Optimal (min)
Avg Response Time	Low (≤ quantum)	High for long jobs	Low for short jobs
Fairness	High	Medium	Low (starvation risk)
Starvation	None	Possible	Likely (long jobs)
Context Switches	Many (quantum)	Few	Many (arrivals)
Practicality	High	Medium	Medium
Best Use Case	Interactive / OS	Batch / known bursts	Batch / min avg WT

7.3 Effect of the Time Quantum on Round Robin

With the selected quantum q=3 and n=5 processes:

- Maximum first-response wait ≤ 3 × (5–1) = 12 time units
- Estimated mid-run preemptions ≥ 7 (processes whose burst > 3)
- A smaller quantum makes RR more responsive but increases context-switch overhead
- A quantum equal to the maximum burst degrades RR to FCFS
- The optimal quantum trades responsiveness against throughput for the specific workload

The quantum is the single tunable parameter in RR. Unlike SJF and SRTF, which derive their scheduling decisions purely from burst times, RR's behaviour is entirely controlled by this constant. A practitioner must profile the expected job lengths and response-time requirements before selecting a quantum for a production system.

8. Fairness & Starvation Discussion

8.1 Fairness Index (Jain's)

Jain's Fairness Index measures how evenly waiting time is distributed across all processes. A value of 1.0 means perfectly equal waiting times; values closer to 0 indicate that some processes wait much more than others.

```
Formula: J = (Σw■)² / (n × Σw■²)
```

Algorithm	Fairness Index (Jain's)	Starvation Risk
RR	★ 0.9049	None (guaranteed)
SJF	0.7246	Medium
SRTF	0.5082	High

On this workload, RR achieves the highest fairness index (0.9049), reflecting its round-robin rotation guarantee. SRTF (0.5082) can rank lower than RR because it continuously reorders the queue based on remaining times, creating unequal waiting times for longer processes.

8.2 Starvation Analysis

Round Robin

Starvation is **impossible** under RR. Every process in the ready queue is served within at most (n–1) × quantum time units. Even if new processes arrive continuously, existing processes cannot be bypassed indefinitely.

SJF (Non-Preemptive)

Starvation is **possible** but bounded per execution. Once a long process starts running it cannot be interrupted, so it will eventually complete. However, if new short processes arrive every time the CPU becomes free, the long process may never reach the front of the scheduling decision.

SRTF (Preemptive SJF)

Starvation risk is **highest** of the three. A long process can be continuously preempted every time a shorter process arrives, potentially waiting for arbitrarily long periods. This is the fundamental theoretical weakness of SRTF for long-running processes.

9. Conclusion & Recommendations

9.1 Per-Metric Verdict (Basic Mixed Workload)

Metric	Winner	Explanation
Lowest Avg Wait Time	SRTF	7.20 — theoretically optimal for preemptive scheduling
Lowest Avg Turnaround	SRTF	SRTF: 13.20 SJF: 16.00
Lowest Avg Response	RR	4.40 — quantum bounds first-access time
Most Fair	RR	Jain index 0.9049 — no starvation possible
Least Starvation	RR	Circular queue guarantees every process is served

9.2 When to Use Each Algorithm

Round Robin (RR)

Use when: Interactive operating systems, desktop environments, web servers handling many simultaneous users, any system where fairness and predictable response times are more important than raw throughput.

Avoid when: Pure batch processing where minimising total completion time matters most.

SJF — Non-Preemptive

Use when: Batch processing systems where job lengths are known in advance, and maximising throughput of short jobs is the priority. Useful as an approximation when execution times can be estimated from history.

Avoid when: Interactive systems where processes should not be forced to wait for long jobs to complete.

SRTF — Preemptive SJF

Use when: High-throughput batch environments where preemption is acceptable, burst times are estimable, and minimising average waiting time is the primary objective. Most efficient algorithm by the WT metric.

Avoid when: Systems with long-running processes that must not be starved, or where context-switch overhead is expensive.

9.3 Final Summary

No single scheduling algorithm is universally superior. The choice depends entirely on the operational requirements of the target system:

- If **fairness and bounded response time** are the priority → **Round Robin**
- If **throughput of short batch jobs** is the priority (non-preemptive) → **SJF**
- If **minimum average waiting time** is the priority (preemption allowed) → **SRTF**
- In practice, modern OSes use *multi-level feedback queues* that combine properties of all three: RR within priority levels, with dynamic priority adjustment that approximates SRTF behaviour over time.

The simulator presented in this project provides an accessible, interactive platform for exploring these trade-offs empirically. Students can modify datasets, adjust the quantum, and immediately observe the effect on all three algorithms' Gantt charts, per-process metrics, and the auto-generated comparative analysis — making it a valuable learning tool for Operating Systems coursework.